Name	Vedant Dinkar Patil
UID no.	2024300170
Experiment No.	Exp3

AIM:	To implement AND Gate using McCulloch Pitts Neuron and Design Half and full adder
Theory:	The McCulloch-Pitts (M-P) neuron is a fundamental computational model that processes binary inputs by summing weighted connections and firing an output of one only when the net excitation meets or exceeds a predetermined threshold. An AND gate is directly implemented using a single M-P neuron with unit excitatory weights assigned to both binary inputs and an activation threshold of two, ensuring it fires exclusively when both inputs are simultaneously active. Building upon this logic, a Half Adder computes the addition of two single-bit inputs by deploying an M-P AND gate to resolve the Carry, while the linearly non-separable Sum (XOR operation) is synthesized through a multi-neuron network where intermediate neurons with inhibitory connections isolate complementary bit pairs and feed into an M-P OR neuron configured with a threshold of one. To handle carry propagation across multiple bit positions, a Full Adder cascades two Half Adders: the first calculates an intermediate sum and carry from the primary data bits, the second adds the incoming carry bit to this intermediate sum to produce the final Sum, and a terminating M-P OR neuron combines the carry outputs from both stages to yield the final Carry Out.

PROGRAM:

AND GATE ::

```
#include <iostream>
using namespace std;

int main()
{
   int x1[] = {0, 0, 1, 1};
   int x2[] = {0, 1, 0, 1};
   int expected[] = {0, 0, 0, 1};
   int p = 0;
   int output;
   bool found = false;

   for (int w1 = 0; w1 <= 1 && !found; w1++)
   {
      for (int w2 = 0; w2 <= 1 && !found; w2++)
      {
         for (int theta = 0; theta <= 4 && !found; theta++)
         {
            cout << "\n---------------------------------\n";
            cout << "Trying: w1 = " << w1
                 << ", w2 = " << w2
                 << ", theta = " << theta
                 << ", p = " << p << endl;

            bool correct = true;
            for (int i = 0; i < 4; i++)
            {
               int n = (x1[i] * w1) + (x2[i] * w2);
               int calculated = n - p;

               if (calculated >= theta)
                  output = 1;
               else
                  output = 0;

               cout << "Input (" << x1[i] << "," << x2[i] << ")"
                    << " -> n = " << n
                    << ", theta = " << theta
                    << ", calculated = " << calculated
```

```
                    << ", output = " << output
                    << ", expected = " << expected[i];

                if (output == expected[i])
                {
                    cout << " <--- Correct";
                }
                else
                {
                    cout << " <--- WRONG";
                    correct = false;
                }

                cout << endl;
            }

            if (correct)
            {
                cout << "\n==================================\n";
                cout << "FINAL SOLUTION \n";
                cout << "w1 = " << w1 << endl;
                cout << "w2 = " << w2 << endl;
                cout << "theta = " << theta << endl;
                cout << "p = " << p << endl;
                cout << "==================================\n";

                found = true;
            }
            else
            {
                cout << "This combination does not give AND output.\n";
            }
        }
    }
}

if (!found)
{
    cout << "\nNo solution found.\n";
}
```

```
    return 0;
}
```

FULL ADDER::

```
#include <iostream>
using namespace std;

int main()
{
    // 3 inputs for Full Adder (8 combinations)
    int x1[]  = {0, 0, 0, 0, 1, 1, 1, 1};
    int x2[]  = {0, 0, 1, 1, 0, 0, 1, 1};
    int cin[] = {0, 1, 0, 1, 0, 1, 0, 1};
    int p = 0;

    // ------------------------------------------------------------
    // STEP 1: INTERMEDIATE SUM S1 = x1 XOR x2
    // ------------------------------------------------------------
    int s1[8];
    for (int i = 0; i < 8; i++)
    {
        // Half adder XOR: (x1 AND NOT x2) OR (NOT x1 AND x2)
        int z1 = ((x1[i] * 1) + (x2[i] * -1) >= 1) ? 1 : 0;
        int z2 = ((x1[i] * -1) + (x2[i] * 1) >= 1) ? 1 : 0;
        s1[i]  = ((z1 * 1) + (z2 * 1) >= 1) ? 1 : 0;
    }

    // ------------------------------------------------------------
    // STEP 2: FINDING FINAL SUM = S1 XOR Cin
    // ------------------------------------------------------------
    cout << "STEP 1: FINDING FINAL SUM (s1 XOR cin)\n";

    // Intermediate z_sum1 = s1 * cin_bar
    bool zSum1Found = false;
    int w_s1_1 = 0, w_cin_1 = 0, theta_sum1 = 0;

    for (int w1 = -1; w1 <= 1 && !zSum1Found; w1++)
    {
```

```
    for (int w2 = -1; w2 <= 1 && !zSum1Found; w2++)
    {
        for (int theta = 0; theta <= 2 && !zSum1Found; theta++)
        {
            bool correct = true;
            for (int i = 0; i < 8; i++)
            {
                int n = (s1[i] * w1) + (cin[i] * w2) - p;
                int out = (n >= theta) ? 1 : 0;
                int expected = (s1[i] == 1 && cin[i] == 0) ? 1 : 0;
                if (out != expected) correct = false;
            }
            if (correct)
            {
                w_s1_1 = w1;
                w_cin_1 = w2;
                theta_sum1 = theta;
                zSum1Found = true;
            }
        }
    }
}

// Intermediate z_sum2 = s1_bar * cin
bool zSum2Found = false;
int w_s1_2 = 0, w_cin_2 = 0, theta_sum2 = 0;

for (int w1 = -1; w1 <= 1 && !zSum2Found; w1++)
{
    for (int w2 = -1; w2 <= 1 && !zSum2Found; w2++)
    {
        for (int theta = 0; theta <= 2 && !zSum2Found; theta++)
        {
            bool correct = true;
            for (int i = 0; i < 8; i++)
            {
                int n = (s1[i] * w1) + (cin[i] * w2) - p;
                int out = (n >= theta) ? 1 : 0;
                int expected = (s1[i] == 0 && cin[i] == 1) ? 1 : 0;
                if (out != expected) correct = false;
```

```
            }
            if (correct)
            {
               w_s1_2 = w1;
               w_cin_2 = w2;
               theta_sum2 = theta;
               zSum2Found = true;
            }
         }
      }
   }

   // Combine z_sum1 OR z_sum2 to get final SUM
   int sum[8];
   for (int i = 0; i < 8; i++)
   {
      int z_a = ((s1[i] * w_s1_1) + (cin[i] * w_cin_1) >= theta_sum1) ? 1 :
0;
      int z_b = ((s1[i] * w_s1_2) + (cin[i] * w_cin_2) >= theta_sum2) ? 1 :
0;
      sum[i]  = ((z_a * 1) + (z_b * 1) >= 1) ? 1 : 0;
   }

   cout << "SUM weights found successfully.\n";

   // ------------------------------------------------------------
   // STEP 3: FINDING CARRY OUT (Cout)
   // Cout = 1 when at least two of (x1, x2, cin) are 1
   // ------------------------------------------------------------
   cout << "\nSTEP 2: FINDING CARRY OUT (Cout)\n";

   bool coutFound = false;
   int c_w1 = 0, c_w2 = 0, c_w3 = 0, cout_theta = 0;

   for (int w1 = 0; w1 <= 1 && !coutFound; w1++)
   {
      for (int w2 = 0; w2 <= 1 && !coutFound; w2++)
      {
         for (int w3 = 0; w3 <= 1 && !coutFound; w3++)
         {
```

```
            for (int theta = 0; theta <= 3 && !coutFound; theta++)
            {
                bool correct = true;
                for (int i = 0; i < 8; i++)
                {
                    int n = (x1[i] * w1) + (x2[i] * w2) + (cin[i] * w3) - p;
                    int out = (n >= theta) ? 1 : 0;

                    // Expected Cout is 1 if sum of inputs >= 2
                    int expected = (x1[i] + x2[i] + cin[i] >= 2) ? 1 : 0;

                    if (out != expected)
                        correct = false;
                }

                if (correct)
                {
                    c_w1 = w1;
                    c_w2 = w2;
                    c_w3 = w3;
                    cout_theta = theta;
                    coutFound = true;

                    cout << "*** Cout SOLUTION FOUND ***\n";
                    cout << "w1 = " << c_w1 << ", w2 = " << c_w2
                        << ", w3 = " << c_w3 << ", theta = " << cout_theta <<
endl;
                }
            }
        }
    }
}

int cout_val[8];
for (int i = 0; i < 8; i++)
{
    int n = (x1[i] * c_w1) + (x2[i] * c_w2) + (cin[i] * c_w3);
    cout_val[i] = (n >= cout_theta) ? 1 : 0;
}
```

```
    // ------------------------------------------------------------
    // STEP 4: FINAL FULL ADDER TRUTH TABLE
    // ------------------------------------------------------------
    cout << "\n-------------------------------------\n";
    cout << "FINAL FULL ADDER TRUTH TABLE\n";
    cout << "-------------------------------------\n";
    cout << "x1   x2   Cin   |   Sum   Cout\n";
    cout << "-----------------------------\n";

    for (int i = 0; i < 8; i++)
    {
        cout << x1[i] << "    "
             << x2[i] << "    "
             << cin[i] << "     |    "
             << sum[i] << "     "
             << cout_val[i] << endl;
    }

    cout << "=============================================\n";

    return 0;
}
```

Output:

```
students@students-HP-280-G3-MT:~/Desktop/2024300170_Vedant$ g++ AND_Gate.cpp -o AND_Gate
students@students-HP-280-G3-MT:~/Desktop/2024300170_Vedant$ ./AND_Gate

--------------------------------
Trying: w1 = 0, w2 = 0, theta = 0, p = 0
Input (0,0) -> n = 0, theta = 0, calculated = 0, output = 1, expected = 0 <--- WRONG
Input (0,1) -> n = 0, theta = 0, calculated = 0, output = 1, expected = 0 <--- WRONG
Input (1,0) -> n = 0, theta = 0, calculated = 0, output = 1, expected = 0 <--- WRONG
Input (1,1) -> n = 0, theta = 0, calculated = 0, output = 1, expected = 1 <--- Correct
This combination does not give AND output.

--------------------------------
Trying: w1 = 0, w2 = 0, theta = 1, p = 0
Input (0,0) -> n = 0, theta = 1, calculated = 0, output = 0, expected = 0 <--- Correct
Input (0,1) -> n = 0, theta = 1, calculated = 0, output = 0, expected = 0 <--- Correct
Input (1,0) -> n = 0, theta = 1, calculated = 0, output = 0, expected = 0 <--- Correct
Input (1,1) -> n = 0, theta = 1, calculated = 0, output = 0, expected = 1 <--- WRONG
This combination does not give AND output.

--------------------------------
Trying: w1 = 0, w2 = 0, theta = 2, p = 0
Input (0,0) -> n = 0, theta = 2, calculated = 0, output = 0, expected = 0 <--- Correct
Input (0,1) -> n = 0, theta = 2, calculated = 0, output = 0, expected = 0 <--- Correct
Input (1,0) -> n = 0, theta = 2, calculated = 0, output = 0, expected = 0 <--- Correct
Input (1,1) -> n = 0, theta = 2, calculated = 0, output = 0, expected = 1 <--- WRONG
This combination does not give AND output.

--------------------------------
Trying: w1 = 0, w2 = 0, theta = 3, p = 0
Input (0,0) -> n = 0, theta = 3, calculated = 0, output = 0, expected = 0 <--- Correct
Input (0,1) -> n = 0, theta = 3, calculated = 0, output = 0, expected = 0 <--- Correct
Input (1,0) -> n = 0, theta = 3, calculated = 0, output = 0, expected = 0 <--- Correct
Input (1,1) -> n = 0, theta = 3, calculated = 0, output = 0, expected = 1 <--- WRONG
This combination does not give AND output.

--------------------------------
```

```
Trying: w1 = 1, w2 = 0, theta = 4, p = 0
Input (0,0) -> n = 0, theta = 4, calculated = 0, output = 0, expected = 0 <--- Correct
Input (0,1) -> n = 0, theta = 4, calculated = 0, output = 0, expected = 0 <--- Correct
Input (1,0) -> n = 1, theta = 4, calculated = 1, output = 0, expected = 0 <--- Correct
Input (1,1) -> n = 1, theta = 4, calculated = 1, output = 0, expected = 1 <--- WRONG
This combination does not give AND output.

---------------------------------
Trying: w1 = 1, w2 = 1, theta = 0, p = 0
Input (0,0) -> n = 0, theta = 0, calculated = 0, output = 1, expected = 0 <--- WRONG
Input (0,1) -> n = 1, theta = 0, calculated = 1, output = 1, expected = 0 <--- WRONG
Input (1,0) -> n = 1, theta = 0, calculated = 1, output = 1, expected = 0 <--- WRONG
Input (1,1) -> n = 2, theta = 0, calculated = 2, output = 1, expected = 1 <--- Correct
This combination does not give AND output.

---------------------------------
Trying: w1 = 1, w2 = 1, theta = 1, p = 0
Input (0,0) -> n = 0, theta = 1, calculated = 0, output = 0, expected = 0 <--- Correct
Input (0,1) -> n = 1, theta = 1, calculated = 1, output = 1, expected = 0 <--- WRONG
Input (1,0) -> n = 1, theta = 1, calculated = 1, output = 1, expected = 0 <--- WRONG
Input (1,1) -> n = 2, theta = 1, calculated = 2, output = 1, expected = 1 <--- Correct
This combination does not give AND output.

---------------------------------
Trying: w1 = 1, w2 = 1, theta = 2, p = 0
Input (0,0) -> n = 0, theta = 2, calculated = 0, output = 0, expected = 0 <--- Correct
Input (0,1) -> n = 1, theta = 2, calculated = 1, output = 0, expected = 0 <--- Correct
Input (1,0) -> n = 1, theta = 2, calculated = 1, output = 0, expected = 0 <--- Correct
Input (1,1) -> n = 2, theta = 2, calculated = 2, output = 1, expected = 1 <--- Correct

=================================
FINAL SOLUTION
w1 = 1
w2 = 1
theta = 2
p = 0
=================================
students@students-HP-280-G3-MT:~/Desktop/2024300170_Vedant$
```

```
students@students-HP-280-G3-MT:~/Desktop/2024300170_Vedant$ g++ half_Adder.cpp -o half_Adder
students@students-HP-280-G3-MT:~/Desktop/2024300170_Vedant$ ./half_Adder

=====================================
STEP 1: FINDING z1 = x1 * x2_bar
=====================================

-------------------------------------
Trying z1: w1 = -1, w2 = -1, theta = 0, p = 0
Input (0,0) -> n = 0, calculated = 0, output = 1, expected = 0 <-- WRONG
Input (0,1) -> n = -1, calculated = -1, output = 0, expected = 0 <-- Correct
Input (1,0) -> n = -1, calculated = -1, output = 0, expected = 1 <-- WRONG
Input (1,1) -> n = -2, calculated = -2, output = 0, expected = 0 <-- Correct
This combination does not give z1.

-------------------------------------
Trying z1: w1 = -1, w2 = -1, theta = 1, p = 0
Input (0,0) -> n = 0, calculated = 0, output = 0, expected = 0 <-- Correct
Input (0,1) -> n = -1, calculated = -1, output = 0, expected = 0 <-- Correct
Input (1,0) -> n = -1, calculated = -1, output = 0, expected = 1 <-- WRONG
Input (1,1) -> n = -2, calculated = -2, output = 0, expected = 0 <-- Correct
This combination does not give z1.

-------------------------------------
Trying z1: w1 = -1, w2 = -1, theta = 2, p = 0
Input (0,0) -> n = 0, calculated = 0, output = 0, expected = 0 <-- Correct
Input (0,1) -> n = -1, calculated = -1, output = 0, expected = 0 <-- Correct
Input (1,0) -> n = -1, calculated = -1, output = 0, expected = 1 <-- WRONG
Input (1,1) -> n = -2, calculated = -2, output = 0, expected = 0 <-- Correct
This combination does not give z1.

-------------------------------------
Trying z1: w1 = -1, w2 = 0, theta = 0, p = 0
Input (0,0) -> n = 0, calculated = 0, output = 1, expected = 0 <-- WRONG
Input (0,1) -> n = 0, calculated = 0, output = 1, expected = 0 <-- WRONG
Input (1,0) -> n = -1, calculated = -1, output = 0, expected = 1 <-- WRONG
Input (1,1) -> n = -1, calculated = -1, output = 0, expected = 0 <-- Correct
This combination does not give z1.
```

```
--------------------------------------
Trying z1: w1 = 0, w2 = 1, theta = 1, p = 0
Input (0,0) -> n = 0, calculated = 0, output = 0, expected = 0 <-- Correct
Input (0,1) -> n = 1, calculated = 1, output = 1, expected = 0 <-- WRONG
Input (1,0) -> n = 0, calculated = 0, output = 0, expected = 1 <-- WRONG
Input (1,1) -> n = 1, calculated = 1, output = 1, expected = 0 <-- WRONG
This combination does not give z1.

--------------------------------------
Trying z1: w1 = 0, w2 = 1, theta = 2, p = 0
Input (0,0) -> n = 0, calculated = 0, output = 0, expected = 0 <-- Correct
Input (0,1) -> n = 1, calculated = 1, output = 0, expected = 0 <-- Correct
Input (1,0) -> n = 0, calculated = 0, output = 0, expected = 1 <-- WRONG
Input (1,1) -> n = 1, calculated = 1, output = 0, expected = 0 <-- Correct
This combination does not give z1.

--------------------------------------
Trying z1: w1 = 1, w2 = -1, theta = 0, p = 0
Input (0,0) -> n = 0, calculated = 0, output = 1, expected = 0 <-- WRONG
Input (0,1) -> n = -1, calculated = -1, output = 0, expected = 0 <-- Correct
Input (1,0) -> n = 1, calculated = 1, output = 1, expected = 1 <-- Correct
Input (1,1) -> n = 0, calculated = 0, output = 1, expected = 0 <-- WRONG
This combination does not give z1.

--------------------------------------
Trying z1: w1 = 1, w2 = -1, theta = 1, p = 0
Input (0,0) -> n = 0, calculated = 0, output = 0, expected = 0 <-- Correct
Input (0,1) -> n = -1, calculated = -1, output = 0, expected = 0 <-- Correct
Input (1,0) -> n = 1, calculated = 1, output = 1, expected = 1 <-- Correct
Input (1,1) -> n = 0, calculated = 0, output = 0, expected = 0 <-- Correct

*** z1 SOLUTION FOUND ***
w1 = 1
w2 = -1
theta = 1
p = 0


--------------------------------------
```

```
======================================
FINAL XOR / HALF ADDER RESULT
======================================

First neuron: z1 = x1 * x2_bar
w1 = 1
w2 = -1
theta = 1
p = 0

Second neuron: z2 = x1_bar * x2
w1 = -1
w2 = 1
theta = 1
p = 0

Final neuron: y = z1 OR z2
v1 = 1
v2 = 1
theta = 1
p = 0

--------------------------------------
FINAL TRUTH TABLE
--------------------------------------
x1   x2   z1   z2   y
---------------------
0    0    0    0    0
0    1    0    1    1
1    0    1    0    1
1    1    0    0    0

======================================
students@students-HP-280-G3-MT:~/Desktop/2024300170_Vedant$
```

CONCLUSION:	This experiment successfully demonstrates the foundational capability of the McCulloch-Pitts neuron model to emulate core arithmetic operations by determining optimal synaptic weights and threshold parameters. While linearly separable functions such as the standard AND gate and the multi-input Carry Out logic are resolved using single-neuron thresholding, the non-linear Exclusive-OR operations required for Sum generation are achieved through multi-layered networks utilizing complementary inhibitory pathways. Cascading these units into Half Adder and Full Adder architectures verifies that complex computational circuits can be constructed purely from primitive artificial neurons, validating the theoretical framework through an exhaustive algorithmic search that accurately reproduces the standard arithmetic truth tables.